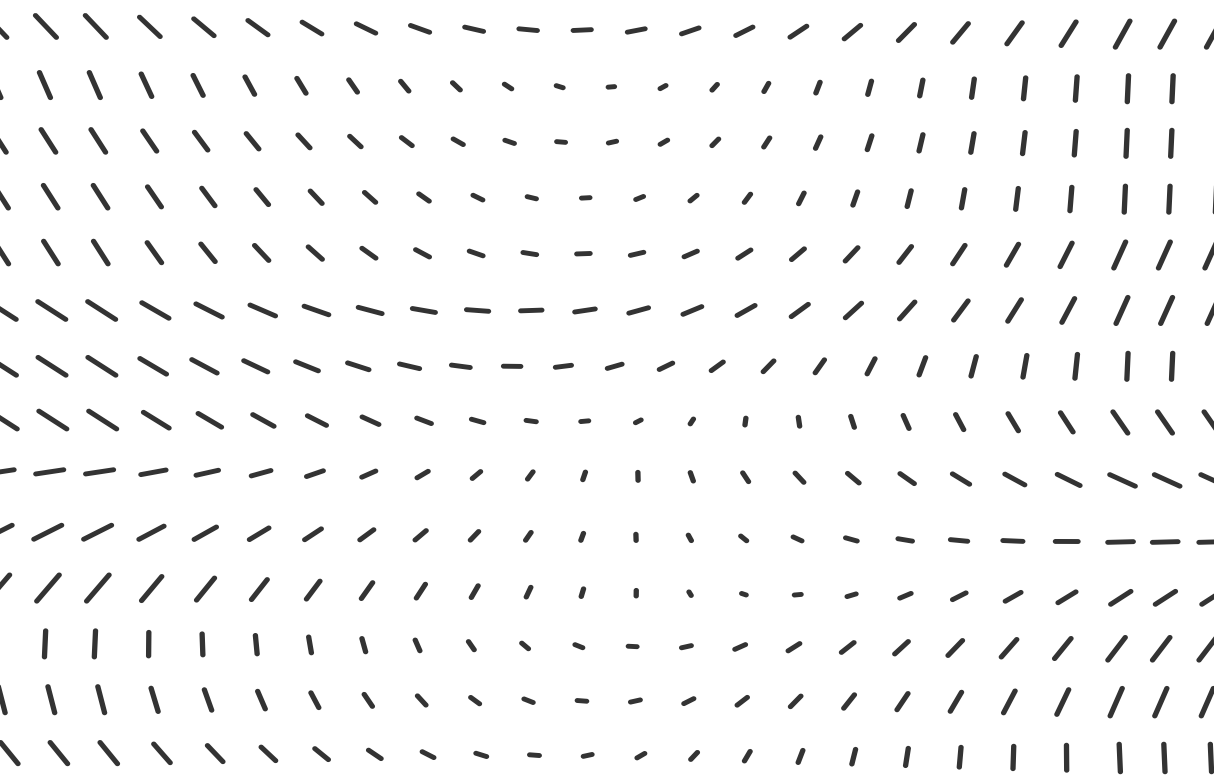
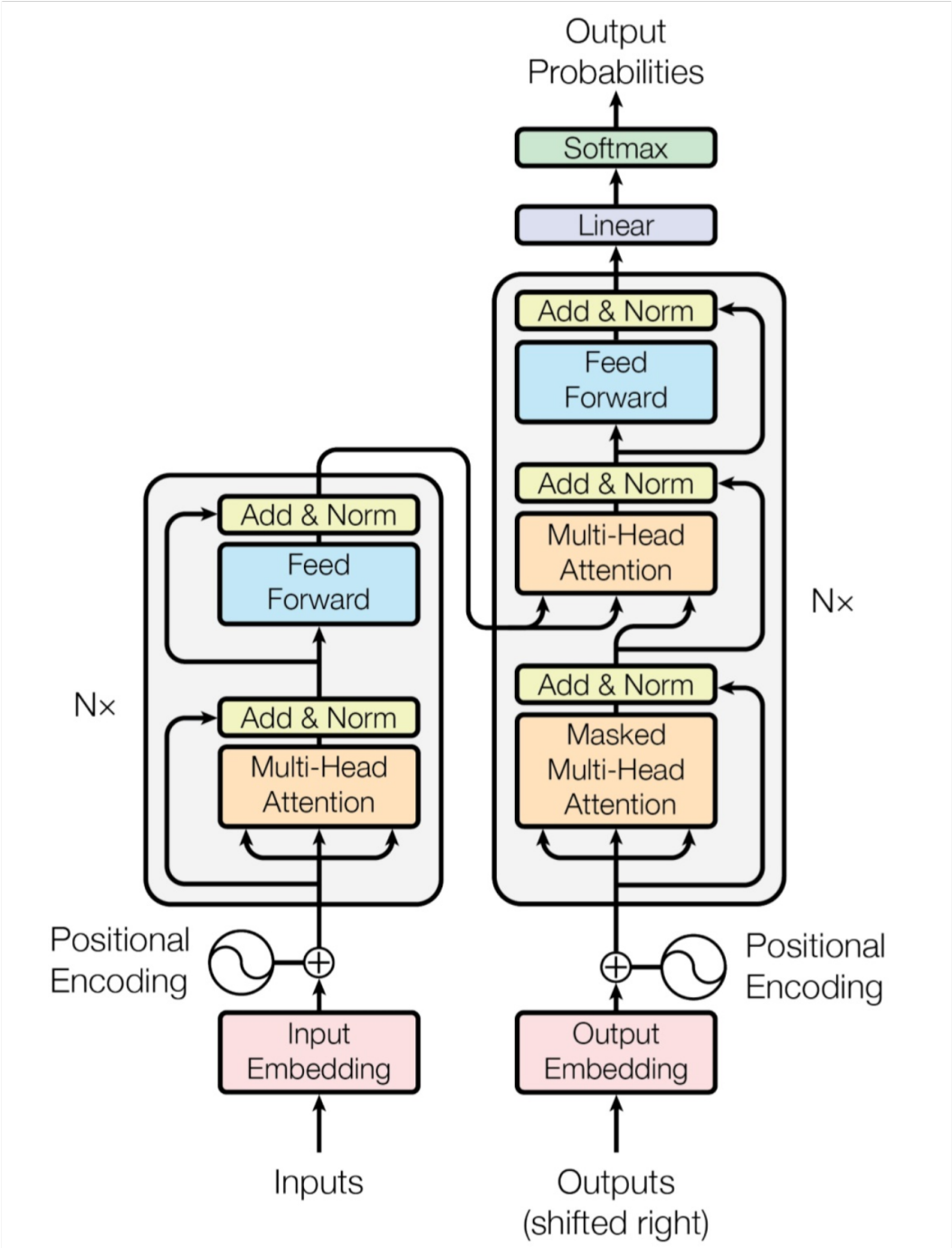
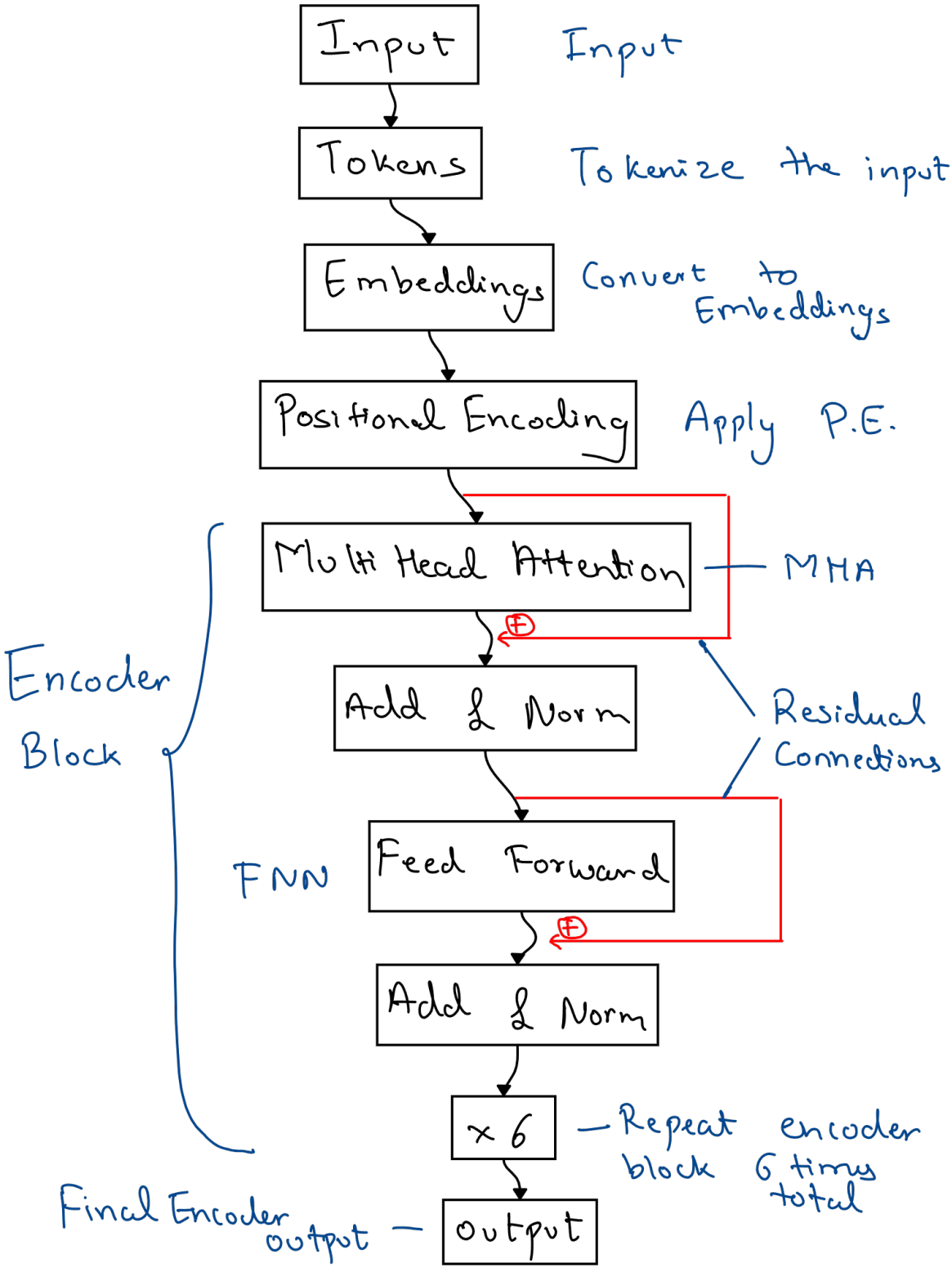
Transformer walkthrough on custom data

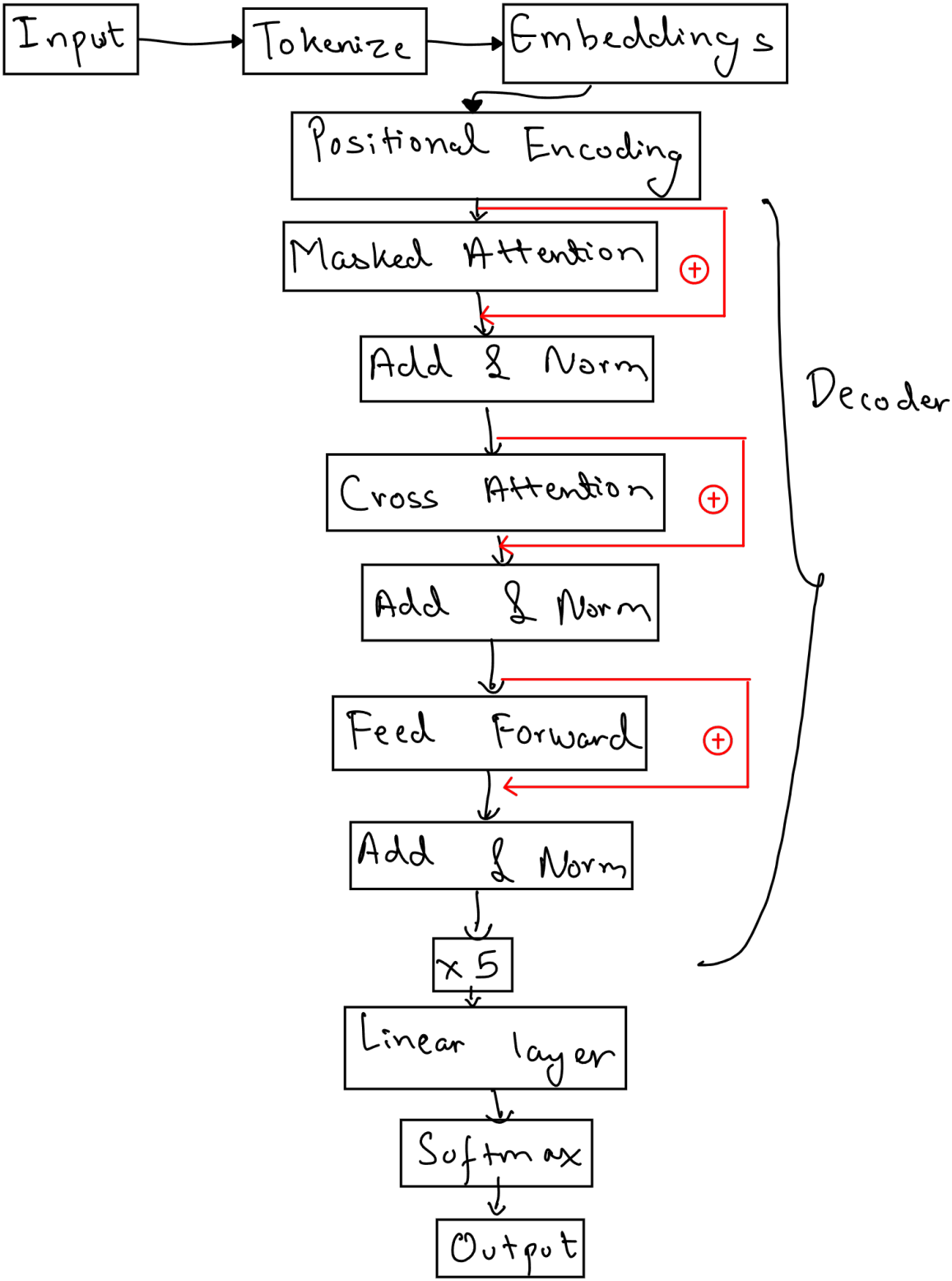
~ Vraj Patel



Transformer Architecture




Input
Input
Tokens
Tokenize the input
Embeddings
Convert to Embeddings
Positional Encoding
Apply P.E.
Multi Head Attention
MHA
Add & Norm
Residual Connections
Encoder Block
FNN
Feed Forward
Add & Norm
x 6
Repeat encoder block 6 times total
Final Encoder output
Output


Input
Tokenize
Embeddings
Positional Encoding
Masked Attention
+
Add & Norm
Cross Attention
+
Add & Norm
Feed Forward
+
Add & Norm
x 5
Decoder
Linear layer
Softmax
Output

Custom Data (Machine Translation)

Input	Target
Good Morning	Guten Tag
I am hungry	Ich habe hunger

Input during Inference :

I am Good

Expected output :

Ich habe guten

During this entire process, our main point is to understand how the data flows through blocks, how are tensors computed and changes shape and How does it predict. Also to understand the concept of KV - Cache.

I will follow exact code line by line starting from train func.

train(model, train_loader, test_loader, criterion, optimizer, device, config)

↓

Epoch = 1

loss = 0

for batch in train_loader

lets also build config.py

```
Config = {
    'd_model' = 4,
    'num_heads' = 2,
    'num_blocks' = 6,
    'src_max_seq_len' = 3,
    'tgt_max_seq_len' = 3,
    'src_vocab_size' = 5,
    'tgt_vocab_size' = 5,
    'max_len' = 6
    'batch_size' = 1
    'learning_rate' = 0.0001,
    'epochs' = 1,
    'path' = "path"
}
```

Tokenization. {Word level}

English		German	
UNK	: 0	UNK	: 0
PAD	: 1	PAD	: 1
SOS	: 2	SOS	: 2
EOS	: 3	EOS	: 3
Good	: 4	Guten	: 4
Morning	: 5	tag	: 5
I	: 6	Ich	: 6
am	: 7	habe	: 7
good	: 8	Hunger	: 8

Therefore : dataset (with padding) + EOS + SOS

Eng	German
2 4 5 1 3	2 4 5 1 3
2 6 7 8 3	2 6 7 8 3

Now back to build_transformer

d_model = 4
num_heads = 2
N = 6
Src_max_seq_len = 3
tgt_max_seq_len = 3
Src_vocab_size = 5
tgt_vocab_size = 5

Eng					german				
2	4	5	1	3	2	4	5	1	3
2	6	7	8	3	2	6	7	8	3

Embeddings

2 = [-0.234, -0.234, 1.592, 0.764]

4 = [. . . . .

⋮

8 . . .

3 . . . ← German Sentence 2

Sentence 1 : x = batch_size, seq_len, d_model
= 2, 3, 4

Positional encoding

x = 2, 3, 4 (before)

↓

2, 3, 4

↓

2 3 4

MHA : Main culprit

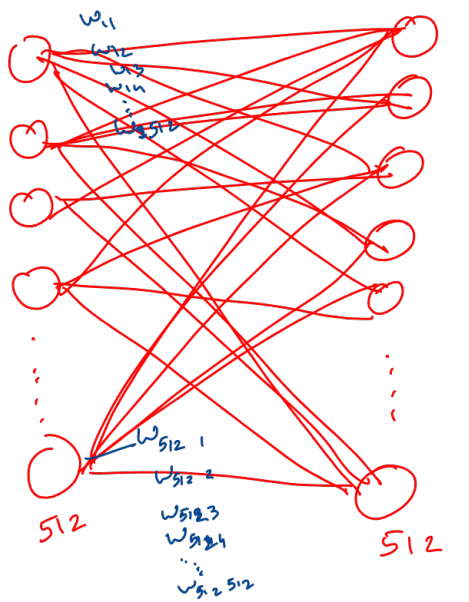
init :

$d_model = 512$

$num_heads = 8$

$d_k = 512/8 = 64$

$w_q = \underbrace{nn.Linear(d_model, d_model)}$



Weight Matrix

$$
\begin{array}{cccccc}
w_{11} & w_{12} & w_{13} & \cdots & w_{51L} & w_{1\,512} \\
w_{21} & \vdots & \vdots & & \vdots & \vdots \\
w_{31} & \vdots & \vdots & & \vdots & \vdots \\
w_{41} & \vdots & \vdots & & \vdots & \vdots \\
\vdots & \vdots & \vdots & & \vdots & \vdots \\
w_{511\,1} & \vdots & \vdots & & \vdots & \vdots \\
w_{512\,1} & & & & &
\end{array}
$$

So its basically:

W_q nn.Linear (d_model, d_model) → Matrix (512, 512)

∴ W_q = Matrix (512, 512)

W_k = Matrix (512, 512)

W_v = Matrix (512, 512)

$\boxed{\text{Init done}}$

forward:

q = W_q(q)

↓

q @ W_q (Mat Multiplication)

Here q: (batch_size, seq_len, d_model)

32, 20, 512) example

W_q = 512 × 512

↓

(32, 20, 512) @ (512 × 512)

for i in range (32)

j in 20

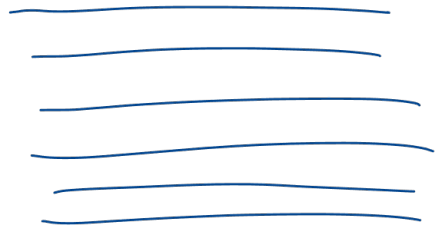
output [i][j] = W_q[i][j].T + bias

both are vectors (1, 512) × (512, 512)

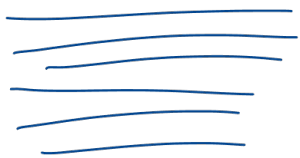
↓

(1, 512)

$i = 1$



$i = 2$



Same and then stack to give tensor

final output : $(32, 20, 512)$

$\therefore$ $q = 32, 20, 512$

$k = 32, 20, 512$

$v = 32, 20, 512$

batch size = 32

q-len = 20

v-len = 20

Now to attention function

$q = q.\text{view}(\text{batch-size}, \text{q-len}, \text{numheads}, d_k).\text{transpose}(1,2)$

$k = \text{''}$

$v = \text{''}$

$= 32, 20, 8, 64 = q = k = v$

Transpose

$= 32, 8, 20, 64$

attention scores (q, k, v, d_k, mask)

$(q @ k.\text{transpose}(-2,-1)) / \sqrt{d_k})$ @ v

q @ k.transpose(-2, -1)

(32, 8, 20, 64)

(32, 8, 20, 64). transpose(-2, -1)

↓

(32, 8, 64, 20)

(32, 8, 20, 64) (32, 8, 64, 20)

@ on 4D tensors does mat multiplication on last 2 dims

So,

(20, 64) (64, 20)

(32, 8 20×20)

↓

32, 8 20×20 / $\sqrt{d_k}$

$d_k = 64$

$\sqrt{64} = 8$

$\sqrt{d_k} = 8$

32, 8, 20, 20 / 8

↓

(32, 8, 20, 20)

softmax (32, 8, 20, 20)

↓

same output

softmax(a, dim=-1)

target

-1 : 20

(32, 8, 20, 20) Q V

(32, 8, 20, 20) (32, 8, 20, 64)

(32, 8, 20, 64)

↓

attention_scores = (32, 8, 20, 64)

attention_scores.transpose(1,2).contiguous().view(batch_size, q_len, d_model)

↓

(32, 8, 20, 64) $\xrightarrow{\text{transpose}(1,2)}$ (32, 20, 8, 64)

↓ .contiguous (simply changes the memory block)

(32, 20, 8, 64)

↓ .view (batch_size, q_len, d_model) (just reshaping tensor)

return W_O (32, 20, 512)

∴ input = output

32, 20, 512 ⟶ 32, 20, 512